

Recipe Parser & AWS Lambda Service Documentation

1 Purpose

This document explains how the Dart-based recipe-parser library and its companion AWS Lambda service work together to transform a raw recipe file (DOCX, PDF, TXT, or PNG) into a fully populated Recipe object ready to be stored and display inside the Interactive Voice for Cooking Recipes app.

2 Architecture at a Glance

Device→Dart Parser: Select & upload file bytes + extension
Dart Parser→Dart Parser: `extractTextFromFile()`
Dart Parser→API Gateway: `POST /parseRecipe {text}`
API Gateway→Lambda: `invoke(parseRecipe)`
Lambda→OpenAI: `ChatCompletion (prompt + text)`
OpenAI→Lambda: JSON recipe draft
Lambda→API Gateway: 200 JSON recipe draft
API Gateway→Dart Parser: JSON recipe draft
Dart Parser→Dart Parser: `convertLLMRecipeToAppRecipe()`
Dart Parser→Device: Recipe object

3 Dart Client Library

3.1 Entry Point `parseRecipeUsingLLMfromBytes`

Step	Responsibility
1. Extract text	<code>extractTextFromFile</code> dispatches to format-specific readers (DOCX ↔ <code>readDocxFileFromBytes</code> , PDF ↔ Syncfusion, PNG ↔ Tesseract-powered <code>Ocr</code>) and returns a <code>List<String></code> of paragraphs.

2. **Build request** Joins paragraphs with `\n`, wraps as `{ "text": <extractedText> }`.
3. **POST** Sends JSON to **API Gateway**
endpoint `https://8f4xlaqil2.execute-api.us-east-1.amazonaws.com/default/parseRecipe` with header `Content-Type: application/json`.
4. **Validate** Non-200 throws; response body is logged verbatim for debugging.
5. **Decode** `jsonDecode` into `Map<String, dynamic>` (schema described in §5.2).

3.2 Schema Converters

- **convertLLMRecipeToAppRecipe** – converts the Lambda/LLM JSON into the app's canonical `Recipe` class, handling:
 - time strings → `Duration` (minutes)
 - servings coercion (`int`, `double`, or numeric `String` → `double`)
 - ingredient strings → structured `RecipeIngredient` via `parseRecipeIngredient`
 - directions list → `List<RecipeStep>` (guarantees ≥1 step)
- **convertParserRecipeToAppRecipe** – legacy path for recipes built locally from DOCX without the LLM.

3.3 Ingredient Parsing Heuristics

`parseRecipeIngredient` recognises **fractions** (`1/2 cup sugar`), **decimals** (`0.75 l water`) and **unit-less items** using two regexes:

```
final fractionRegex = RegExp(r'^(\d+\d+)\s*([a-zA-Z]+)?\s*(.*)$');  
final decimalRegex = RegExp(r'^(\d+(\.\d+)?)\s*([a-zA-Z]+)?\s*(.*)$');
```

Unrecognised lines still map to a valid `RecipeIngredient` with `amount=null` and `displayAmount="0"` so the UI never crashes.

3.4 Multi-format Text Extraction

Format	Method	Notes
DOCX	ZIP-decode then regex <code><w:t></code> tags	No external package needed.
PDF	<code>SyncfusionFlutterPdf</code> -based text extractor	Disposed after use.
TXT	UTF-8 decode.	
PNG	Tesseract OCR service	Returns UTF-8 string.

3.5 OCR (PNG → Text)

When the user uploads a **PNG** (or any bitmap) the client converts it to plain text locally before calling the Lambda so that the payload stays small and the LLM only processes clean text.

Step	Component	What Happens
1	<code>extractTextFromFile()</code> (Dart)	Detects the <code>.png</code> extension and calls <code>readPngFileFromBytes()</code> .
2	<code>readPngFileFromBytes()</code> (Dart)	Passes the raw <code>Uint8List</code> image bytes down to a JavaScript bridge exposed by a <code>WebView</code> (or Flutter → JS interop on the web).
3	<code>_extractText()</code>	Spins up an isolated Tesseract.js worker, loads the requested language pack (default <code>eng</code>), applies any custom OCR parameters, and runs on the image.
4	Return to Dart	The recognised string (<code>res.data.text</code>) is sent back, split on <code>\n</code> , and fed into the normal LLM pipeline

4 AWS Side

4.1 API Gateway

- **Method:** `POST`
- **Path:** `https://8f4xlaqil2.execute-api.us-east-1.amazonaws.com/default/parseRecipe`
- **CORS:** Enabled for `*` (handled via Gateway settings)
- **Timeout:** 20 s

4.2 Lambda Function `parseRecipe`

Property	Value
----------	-------

Runtime	<code>Node.js 22</code>
---------	-------------------------

Memory	128 MB
--------	--------

Timeout	20 s
---------	------

Env Vars	<code>OPENAI_API_KEY</code>
----------	-----------------------------

4.2.1 Handler Outline (pseudo-code)

```
function handler(event):  
  try  
    # Detect HTTP method (works for REST v1 or HTTP v2 APIs)  
    method ← event.requestContext?.http?.method  
              OR event.httpMethod  
              OR ""  
  
    # CORS pre-flight?  
    if method = "OPTIONS":  
      return 204 with CORS headers  
  
    # Reject anything except POST  
    if method ≠ "POST":  
      return 405 "Method Not Allowed"  
  
    # Parse JSON body and extract `text`  
    try  
      payload ← JSON.parse(event.body or "{}")
```

```

    text ← payload.text
  catch parseError
    return 400 "Invalid JSON"

  if text is empty
    return 400 "Missing 'text' field"

  # Build prompt string that instructs LLM to return JSON fields
  prompt ← ""
    You are a recipe parser...
    (list of required keys)
    ...Recipe text:
    "" + text

  # Call OpenAI chat completion
  response ← openai.chat.completions.create(
    model = "gpt-3.5-turbo",
    temperature = 0.2,
    messages = [
      {role:"system", content:"You are a recipe parser that returns only JSON."},
      {role:"user", content:prompt}
    ]
  )

  # Try to parse the LLM output as JSON
  try
    resultJson ← JSON.parse(response.choices[0].message.content)
  catch jsonError
    raise Error("Invalid JSON from LLM")

  # Success – return parsed JSON to caller
  return 200 with CORS headers and body = resultJson

catch anyError
  log error
  return 500 with CORS headers and error details (message + first 3 stack lines)

```

Lambda performs zero parsing; it leaves this to OpenAI and merely passes the response as JSON back to the client.

4.3 Prompt Engineering Details

- **Explicit JSON schema** attached as a system message so the model validates keys (`recipeName`, `ingredientList`, `directions`, ...).
 - **Few-shot examples** improve robustness on noisy OCR output.
 - **Temperature 0.2** for deterministic structure.
-

5 Data Contracts

5.1 Request Body (Client → Lambda)

```
{  
  "text": "<full plain-text recipe>"  
}
```

5.2 Response Body (Lambda → Client)

```
{  
  "recipeName": "String",  
  "description": "String",  
  "ingredientList": [ "String", ... ],  
  "directions": [ "String", ... ],  
  "prepTime": 15,      // minutes (int)  
  "cookTime": 30,      // minutes (int)  
  "totalTime": 45,     // minutes (int)  
  "servings": 4,  
  "equipment": [ "pan", "spatula" ],  
  "nutrition": { "Calories": "250 kcal", ... },  
  // ...optional extra keys  
}
```

6 Error Handling & Logging

- **Network failures** - rethrown after printing stack trace; upstream UI shows toast.
- **Non-200** - body text logged to aid debugging; exception bubbled.

- **JSON decode errors** - guarded with try/catch and detailed log including raw response body.
 - **Regex misses** - functions such as `matchPrepCookTime` emit warnings instead of throwing, ensuring partial recipes still parse.
-